apsara

Date: __-__-____

# CLAUSE

1. WHERE Clause
2. GROUP BY Clause
3. HAVING Clause
4. ORDER BY Clause
5. LIMIT
6. OFFSET

## 2. GROUP BY CLAUSE

Combines rows that have the same values into groups, usually calculate total, averages or counts. We usually use it with aggregate functions. such as SUM(), COUNT(), AVG(), MAX(), MIN() To calculate info for each group.

Ex: In the employee table.

| Employee | Department | Salary |
|---|---|---|
| Ravi | IT | 30,000 |
| priya | HR | 25,000 |
| Arun | IT | 40,000 |
| Meena | HR | 35,000 |
| Kiran | Sales | 20,000 |
| Divya | Sales | 28,000 |



Scanned with OKEN Scanner

apsara

Date: —·—·——

Suppose manager asks " what is the total salary for each department ? "

We could add each departments and salaries manually, but SQL can do it automatically using GROUP BY

The key idea is that SQL makes one group for IT , one for HR and , one for sales , then calculates the salary total separately for each group.

Syntax : SELECT column_name,
aggregate_function (column_name)
FROM table name
GROUP BY column_name;

Ex : SELECT Department, SUM (salary) AS total_salary
FROM Employees
GROUP BY Department;



Scanned with OKEN Scanner

apsara

Date:

## 3. HAVING CLAUSE

Filters groups after GROUP BY has grouped the rows and calculated aggregate values

Suppose we want to display only department whose salary is greater than ₹60,000.

Syntax :
```
SELECT column_name,
    aggregate_function (column_name)
    FROM table_name
    GROUP BY column_name
    HAVING condition;
```

Ex :
```
SELECT Department, SUM (salary)
    FROM Employees
    GROUP BY Department
    HAVING SUM (salary) > 60000;
```

This will display only department salary greater than 60,000

Page No.:

Scanned with OKEN Scanner

| | WHERE | HAVING |
|---|---|---|
| * | Filters individual rows | * Filters groups |
| * | Applied before grouping | * Applied after grouping |
| * | Normally filters using row-level conditions | * can filter using aggregate functions |

- WHERE = filters rows
- HAVING = filter groups

## 4. ORDER BY CLAUSE

Sorts query results in ascending or descending order

syntax : SELECT * FROM table_name
ORDER BY column_name ASC;

Ex : SELECT * FROM Employees
ORDER BY Salary ASC;

ORDER BY is used to sort the results in ascending (ASC) or descending (DESC) order And we can also sort employees names in ord alphabetical order



Scanned with OKEN Scanner

ASC → Alphabetical order ( A to Z )
DESC → Reverse alphabetical order ( z to A )

## 5. LIMIT CLAUSE

This restricts the number of rows returned by a query

Syntax: SELECT * FROM ~~Employees~~ table_name
LIMIT number ;

Ex: SELECT * FROM Employees
LIMIT 2 ;
only 2 employee rows are returned

If we want the two highest paid employees, combine LIMIT with ORDER BY :

Ex: SELECT * FROM Employees
ORDER BY salary DESC
LIMIT 2;

→ LIMIT controls how many rows we receive, not which rows qualify
→ ORDER BY used when we need particular ranking.



Scanned with OKEN Scanner

apsara

Date: —/—/——

6. OFFSET CLAUSE

Skips a specified number of rows before returning results

Syntax: SELECT * FROM table_name
LIMIT number
OFFSET number;

Ex: SELECT * FROM Employees
LIMIT 2
OFFSET 2;

Assuming table's rows are ordered as Ravi, Priya, Arun, Meena, the query skips the 1st 2000 rows and returns

| Name | Department | Salary |
|---|---|---|
| Arun | IT | 40000 |
| Meena | HR | 35000 |

For predictable results, use ORDER BY with OFFSET.



Scanned with OKEN Scanner